

- Secondary Storage.” *ACM Computing Surveys*, June 1994.
- CHEN96** Chen, S., and Towsley, D. “A Performance Evaluation of RAID Architectures.” *IEEE Transactions on Computers*, October 1996.
- DALT96** Dalton, W., et al. *Windows NT Server 4: Security, Troubleshooting, and Optimization*. Indianapolis, IN: New Riders Publishing, 1996.
- DELL00** Dekker, E., and Newcomer, J. *Developing Windows NT Device Drivers: A Programmer's Handbook*. Reading, MA: Addison-Wesley, 2000.
- FRIE96** Friedman, M. “RAID Keeps Going and Going and...” *IEEE Spectrum*, April 1996.
- MEE96a** Mee, C., and Daniel, E. eds. *Magnetic Recording Technology*. New York: McGraw-Hill, 1996.
- MEE96b** Mee, C., and Daniel, E. eds. *Magnetic Storage Handbook*. New York: McGraw-Hill, 1996.
- NG98** Ng, S. “Advances in Disk Technology: Performance Issues.” *Computer*, May 1989.
- PAI00** Pai, V.; Druschel, P.; and Zwaenepoel, W. “IO-Lite: A Unified I/O Buffering and Caching System.” *ACM Transactions on Computer Systems*, February 2000.
- ROSC03** Rosch, W. *The Winn L. Rosch Hardware Bible*. Indianapolis, IN: Que Publishing, 2003.
- SCHW96** Schwaderer, W., and Wilson, A. *Understanding I/O Subsystems*. Milpitas, CA: Adaptec Press, 1996.
- SELT90** Seltzer, M.; Chen, P.; and Ousterhout, J. “Disk Scheduling Revisited *Proceedings, USENIX Winter Technical Conference*, January 1990.
- STAL06** Stallings, W. *Computer Organization and Architecture, 7th ed.* Upper Saddle River, NJ: Prentice Hall, 2006.
- WIED87** Wiederhold, G. *File Organization for Database Design*. New York: McGraw-Hill, 1987.
- WORT94** Worthington, B.; Ganger, G.; and Patt, Y. “Scheduling Algorithms for Modern Disk Drives.” *ACM SIGMETRICS*, May 1994.

11.13 关键术语、复习题和习题

关键术语

块	磁盘组	固定磁盘
面向块的设备	固定头磁盘	可编程 I/O
循环缓冲区	软盘	读/写头
CD-R	间隙	独立磁盘冗余阵列 (RAID)
CD-ROM	硬盘	CD-RW
中断驱动 I/O	可换式磁盘	柱面
输入/输出 (I/O)	旋转延迟	设备 I/O
I/O 缓冲区	扇区	数字化视频光盘 (DVD)
I/O 通道	寻道时间	直接存储器访问 (DMA)
I/O 处理器	面向流的设备	磁盘存取时间
逻辑 I/O	磁道	磁盘高速缓存
磁盘	传送时间	活动头磁盘

复习题

- 列出并简单定义执行 I/O 的三种技术。
- 逻辑 I/O 和设备 I/O 有什么区别？
- 面向块的设备 and 面向流的设备有什么区别？各举一些例子。
- 为什么希望用双缓冲而不是单缓冲来提高 I/O 的性能？
- 在磁盘读或写时有哪些延迟因素？
- 简单定义图 11.7 中描述的磁盘调度策略。
- 简单定义 7 个 RAID 级别。

11.8 典型的磁盘扇区大小是多少？

习题

- 11.1** 考虑一个程序访问一个 I/O 设备，并比较无缓冲的 I/O 和使用缓冲区的 I/O。说明使用缓冲区最多可以减少 2 倍的运行时间。
- 11.2** 思考下面的问题，你会注意到本章讨论的磁盘调度算法中存在一些“边界情况”需要考虑。
- 设计并描述一组寻道请求序列，使得下述磁盘调度算法产生相同（或相似）的执行效率：FCFS、SSTF、随机算法、SCAN（使用 LOOK）。
 - 设计并描述一个寻道请求序列，使得 SCAN 和 C-SCAN（使用 LOOK）算法产生相同（或相似）的执行效率。
- 11.3** a) 使用与表 11.2 类似的方式，分析下列磁道请求序列：27, 129, 110, 186, 147, 41, 10, 64, 120。假设磁头最初定位在磁道 100 处，并且沿着磁道号减小的方向移动。
- b) 如果假设磁头沿着磁道号增大的方向移动，请给出同样的分析。
- 11.4** 考虑一个磁盘，有 N 个磁道，磁道号从 0 到 $(N-1)$ ，并且假设请求的扇区随机地均匀分布在磁盘上。现在要计算一次寻道平均跨越的磁道数。
- 首先，计算当磁头位于磁道 i 时，寻道长度为 j 的可能性。（提示：这是一个关于确定所有组合数目的问题，所有磁道位置作为寻道目标的可能性是相等的。）
 - 接下来计算寻道长度为 K 的可能性。（提示：这包括所有移动了 K 个磁道的可能性之和。）
 - 使用下面计算期望值的公式，计算一次寻道平均跨越的磁道数目：

$$E[x] = \sum_{i=0}^{N-1} i \sum \text{Pr}[x=i]$$

提示：使用 $\sum_{i=1}^n i = \frac{n(n+1)}{2}$; $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

d) 说明当 N 比较大时，一次寻道平均跨越的磁道数接近 $N/3$ 。

- 11.5** 下面的公式适用于高速缓存存储器和磁盘高速缓存：

$$T_S = T_C + M \times T_D$$

请把这个公式推广到 N 级存储器结构，而不是仅仅 2 级。

- 11.6** 对基于频率的置换算法（见图 11.9），定义 F_{new} 、 F_{middle} 和 F_{old} 分别为包含新区、中间区和老区的高速缓存片断，显然 $F_{\text{new}} + F_{\text{middle}} + F_{\text{old}} = 1$ 。如果有

a) $F_{\text{old}} = 1 - F_{\text{new}}$

b) $F_{\text{old}} = 1 / (\text{高速缓存大小})$

请分别描述该策略。

- 11.7** 如果磁盘中扇区大小固定为每扇区 512 字节，并且每磁道 128 个扇区，每面 130 个磁道，一共有 12 个可用的面，计算存储 90000 条 200 比特长的逻辑记录需要多少磁盘空间（扇区、磁道和面）。忽略文件头记录和磁道索引，并假设记录不能跨越两个扇区。
- 11.8** 考虑 11.7 中提到的磁盘系统。假设磁盘转速是 1200 转/分，磁盘控制器每旋转一圈可以将一个扇区读入其内部缓冲区，接着操作系统以字节为单位读取这些数据，每读取一个字节，磁盘控制器都会产生一个中断。
- 如果中断服务例程处理每个中断的时间是 1.8 微秒，那么系统读入整个扇区需要花多少时间？（不考虑寻道所需要花费的时间）
 - 在系统读取数据的过程中，操作系统可以用于处理其他进程的时间是多少？相对于读取磁盘的总传输时间，所占的百分比是多少？
- 11.9** 再次考虑习题 11.7 和 11.8 中的磁盘系统。假设磁盘控制器和系统内存之间的数据传输采用 DMA 方式，总线速度为 2MB/秒。在该条件下，系统读入整个扇区需要花费多少时间？在这段时间内，操作系统可以用于处理其他进程的时间是多少？
- 11.10** 一个 32 位计算机有两个选择通道和一个多路通道，每个选择通道支持两个磁盘和两个磁带部件。多路通道有两个行式打印机、两个卡片阅读机，并连接着 10 个 VDT 终端。假设有以下的传送率：

磁盘驱动器	800KB/s
磁带驱动器	200KB/s
行式打印机	6.6KB/s
卡片阅读器	1.2KB/s
VDT	1KB/s

估计系统中最大总的 I/O 传送率为多少？

- 11.11 当条带的大小比 I/O 请求的大小时，磁盘条带化显然可以提高数据传送率。同样，相对于单个的大磁盘，由于 RAID 0 可以并行处理多个 I/O 请求，显然它可以提高性能。但是，对于后一种情况，磁盘条带化还有必要存在吗？也就是说，相对于没有条带化的磁盘阵列，磁盘条带化可以提高 I/O 请求速率的性能吗？
- 11.12 现有一个 RAID 磁盘阵列，包含四个磁盘，每个磁盘大小都是 200GB。试问在 RAID 级分别为 0,1,2,3,4,5,6 时，该磁盘阵列的有效存储容量是多少？

附录 11A 磁盘存储设备

磁盘

磁盘是由表面涂有磁性物质的金属或塑料构成的圆形盘片。数据被记录在磁盘中，然后通过一个称做磁头的导体线圈从磁盘中取出。在进行读操作或写操作期间，磁头是固定的，磁盘在它下面旋转。

写机制基于通过线圈的电流会产生磁场的原理。电磁脉冲被送到磁头，产生的磁模式被记录在磁头下面的磁盘面中，正负电流会产生不同的磁模式。读机制基于这一事实：相对于线圈移动的磁场会在线圈中产生电流。当盘片在磁头下面经过时，可以产生与以前记录的极性相同的电流。

数据组织和格式化

磁头是一个相对较小的设备，它能够从旋转到其下面的盘片部分中读取或写入数据。这就导致盘片上的数据被组织在一组同心圆中，称为磁道。每个磁道与磁头一样宽，每个盘面上有上千个磁道。

图 11.16 描绘了磁盘数据的布局。相邻的磁道通过间隙被分隔开，这样可以避免或者至少减小由于磁头未对准或磁场之间的干扰引发的错误。

数据按扇区（见图 11.16）从磁盘中向外或向磁盘里传送。通常每个磁道有几百个扇区，它们可以是固定长度的，也可以是可变长度的。对于大多数磁盘驱动器，扇区大小固定为 512 个字节。为避免对系统强加不合理的精度要求，相邻的扇区通过磁道内部（扇区间）的间隙来分隔。

靠近旋转的磁盘中心的一个数据位，通过一个固定点（如读写磁头）时候的速度要慢于磁盘外侧的一个数据位。因此必须找到某种办法来补偿这种速度上的差别，使得磁头可以以相同的速度读取每个比特。这个问题可以通过增加记录在磁盘上信息的比特位之间的间隙来解决。这样信息就可以通过定速旋转的磁盘，以相同的速度扫描，这称为恒定角速度（Constant Angular Velocity, CAV）。图 11.17a 显示了使用 CAV 技术的磁盘布局。磁盘被划分成饼状的扇区以及一系列同心的磁道。使用 CAV 的好处是每块数据可以由磁道和扇区直接定位。为了将磁头从当前的位置移动到指定的位置，只需将磁头短距离移动到指定的磁道，然后等待合适的扇区旋转到磁头下面。CAV 的不足是，磁盘外圈的磁道尽管较长，但是其保存的数据量跟较短的内圈磁道相同。

由于存储密度单位面积的比特数从最外边的磁道向最里面的磁道递增，因此采用一般 CAV 系统的磁盘，其存储容量受限于最内磁道的最大记录密度。为了增加密度，现代磁盘系统使用了一种称为多区记录（Multiple zone recording）的技术，该技术中磁盘表面被划分为多个同心的区域（典型为 16 个）。在一个区域内，每个磁道的比特数是常数。那些远离中心的区域比那些靠近中心的区域含有更多的比特。这使得可以获得更大的存储空间，其代价是更复杂的旋转控制。当磁盘头从一个区域移动到另一个区域的时候，每个比特的长度也随着磁道的变化而变化，这就引起了读写时间的变化。图 11.17b 表示了多区记录的分布情况，其中，每个区域是一个磁道宽。

需要一些方法在一个磁道中定位扇区的位置。很显然，磁道上必须有个开始点以及一个判断每个扇区开始和结束的方法。这些需求可以由磁盘上记录的控制数据来处理。因而，磁盘格式化后会包含一些额外数据，这些数据只能被磁盘驱动器使用，用户是不能访问的。

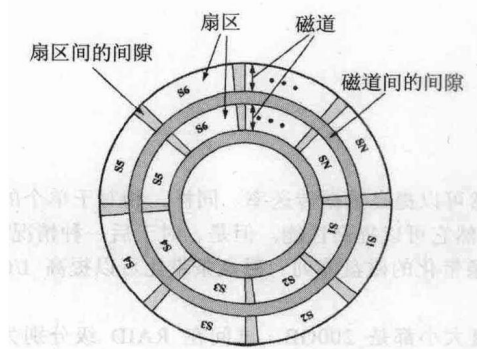


图 11.16 磁盘数据布局

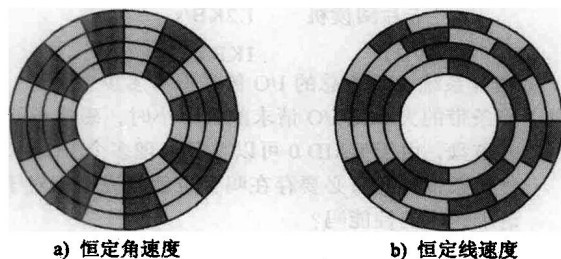


图 11.17 磁盘布局的比较

物理特性

表 11.6 列出不同类型的磁盘之间的主要差别。首先，磁头相对于盘片的径向方向可以是固定的，也可以是可移动的。对于固定头磁盘（fixed-head disk），每个磁道都有一个读/写磁头。所有磁头都安装在一个刚性的磁头臂中，磁头臂可以伸展，跨过所有的磁道。而活动头磁盘（movable-head disk）只有一个读/写磁头，同样，这个磁头也安装在一个磁头臂中。由于磁头必须能够定位在任何一个磁道上，磁头臂可以为这个目的伸展或缩回。

表 11.6 磁盘系统的物理特性

磁头移动 固定头（每个磁道一个） 活动头（每个盘面一个）	盘片 单个盘片 多个盘片
磁盘可移动性 固定磁盘 可换式磁盘	磁头机制 接触（软盘） 固定间隙 空气动力间隙（Winchester）
面 单面 双面	

磁盘本身安装在一个磁盘驱动器中，磁盘驱动器由磁头臂、用于旋转磁盘的轴心和二进制数据输入输出所需要的电子设备组成。固定磁盘（nonremovable disk）永久地安装在磁盘驱动器上，个人计算机中的硬盘就是一个固定磁盘。可换式磁盘（removable disk）可以被移走并用另一个磁盘替换。可换式磁盘的优点是可以通过有限数量的磁盘系统得到无限的数据量。此外，这类磁盘可以从一个计算机系统移到另一个。软盘和 ZIP 磁盘就是可换式磁盘的例子。

对于大多数磁盘，盘片的两面都有磁涂层，称做双面（double sided）。某些比较便宜的磁盘系统使用单面（single-sided）磁盘。

某些磁盘驱动器允许多个盘片（multiple platter）垂直堆叠起来，彼此之间相距 1 英寸，同时提供多个磁头臂。这些盘片称做磁盘组（disk pack），如图 11.18 所示。多盘片磁盘采用活动头，每个盘面一个读/写磁头。所有的磁头被机械地固定在一起，使得它们距磁盘中心的距离相同，并且可以一起移动。因此，在任何时候，所有的磁头都位于与磁盘中心距离相同的磁道上。所有盘片上相对位置相同的磁道的组合称做柱面（cylinder）。例如，图 11.19 中所有用阴影表示的磁道就是一个柱面的一部分。

最后，根据磁头机制可以把磁盘划分成三类。传统上，读/写磁头位于盘片上方一个固定距离处，允许有一空气间隙。另一种极端是在读写操作期间，磁头机制在物理上接触到了介质。这种机制常用于软盘。软盘是一种比较小、比较灵活并且很便宜的磁盘类型。

为理解第三种类型的磁盘，有必要解释一下数据密度与空气间隙的大小之间的关系。为了正确地读或写，磁头必须能够产生或感知到足够的电磁场。磁头越窄，它在工作时必须离盘片越近。磁头比较窄意

意味着磁道比较窄，从而使得数据密度比较大。但是，磁头离磁盘越近，由于杂质或瑕疵发生错误的危险也就越大。Winchester 磁盘的研制进一步推动了这项技术的发展。Winchester 磁头用在密封的、几乎没有杂质的驱动器部件中。它们被设计成在操作时比传统的刚性磁头更接近盘面，因而允许更大的数据密度。磁头实际上是一个空气动力薄膜，当磁盘不动时轻轻地停在盘面上，磁盘旋转时产生的气压足以使这个薄膜升高离开盘面。由此得到的非触式系统可以使用更窄的磁头，可以比传统的刚性磁头距离盘面更近进行操作^①。

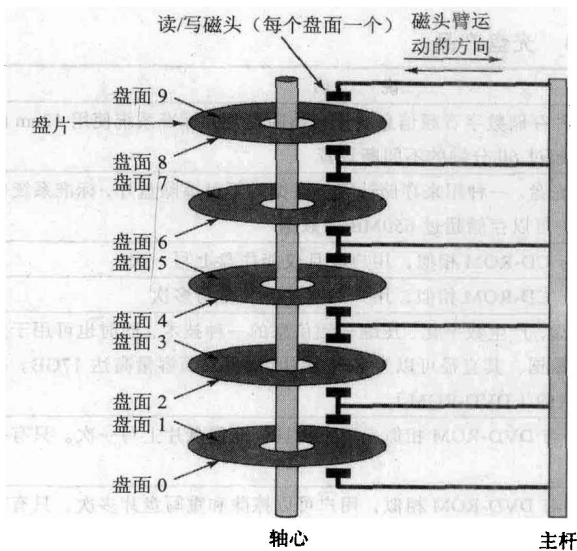


图 11.18 磁盘驱动器的组成

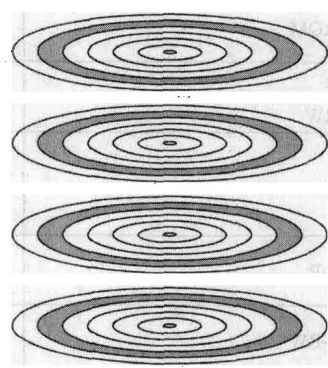


图 11.19 磁道和柱面

表 11.7 给出了当代典型的高性能磁盘的磁盘参数。

表 11.7 典型的磁盘驱动器参数

特 性	Seagate Barracuda 180	Seagate Cheetah X15-36LP	Seagate Barracuda 36ES	Toshiba HDD1242	Hitachi Microdrive
应用	大容量服务器	高性能服务器	入门级台式机	便携	手持设备
容量	181.6GB	36.7GB	18.4GB	5GB	4GB
从磁道到磁道的最小寻道时间	0.8ms	0.3ms	1.0ms	—	1.0ms
平均寻道时间	7.4 ms	3.6 ms	9.5 ms	15 ms	12 ms
轴心速度	7200 r/m	15K r/m	7200 r/m	4200 r/m	3600 r/m
平均旋转延迟	4.17 ms	2 ms	4.17 ms	7.14 ms	8.33 ms
最大传送率	160 MB/s	522~709 MB/s	25 MB/s	66 MB/s	7.2MB/s
每扇区的字节数	512	512	512	512	512
每磁道的扇区数	793	485	600	63	—
每柱面的磁道数(盘面数)	24	8	2	2	2
柱面(盘片一面的磁道数)	24247	18479	29851	10350	—

① 从历史的角度看，术语 Winchester 最早出自 IBM 3340 磁盘模型发布前的编号。3340 磁盘模型是一种可换式磁盘，磁头密封其中。该术语目前指的是任何具有空气动力磁头设计的封装磁盘。Winchester 磁盘通常用于构造个人电脑和工作站，也称为硬盘。

光存储器

1983 年引进的光盘 (CD) 数字声音系统是史上最成功的消费产品之一。CD 是一种不可擦写的磁盘, 一面可以存储超过 60 分钟的声音信息。CD 在商业上的巨大成功推动了价格便宜的光盘存储技术的开发, 从而给计算机数据存储带来了重大的革命。当前, 已经出现了各种各样的光盘系统 (见表 11.8), 下面简单介绍几种。

表 11.8 光盘产品

类 型	说 明
CD	压缩光盘。一种存储数字音频信息的不可擦除盘片, 标准系统使用 12cm 的盘片, 可以记录超过 60 分钟的不间断录音
CD-ROM	只读存储压缩光盘。一种用来存储计算机数据的不可擦除盘片, 标准系统使用 12cm 的盘片, 可以存储超过 650MB 的数据
CD-R	可记录 CD, 与 CD-ROM 相似, 用户能且仅能往盘上写一次
CD-RW	可重写 CD, 与 CD-ROM 相似, 用户可以擦除和重写多次
DVD	数字化视频光盘, 产生数字化、压缩视频信息的一种技术, 同时也可用于其他大容量数字式数据。其直径可以为 8cm 和 12cm, 双面容量高达 17GB。基本的 DVD 是只读的 (DVD-ROM)
DVD-R	可记录 DVD, 与 DVD-ROM 相似, 用户能且仅能往盘片上写一次。只有一面可用
DVD-RW	可重写 DVD, 与 DVD-ROM 相似, 用户可以擦除和重写盘片多次。只有一面可用
HD-DVD	高清 DVD, 提供比普通 DVD 更大的数据存储容量, 使用 405 nm (蓝紫光 blue violet) 激光。单面单层可存储 15GB
蓝光-DVD	类似于 HD-DVD。单面单层可存储 25GB

CD-ROM

音频 CD 和 CD-ROM (光盘只读存储器, Compact Disc Read-Only Memory) 使用的是同一种技术。主要差别是 CD-ROM 具有纠错设备, 确保数据可以正确地光盘传送到计算机。这两种类型的光盘是用同样的方法制作的。光盘是用一种树脂制成的, 如聚碳酸酯。记录的数字化信息 (音乐数据或计算机数据) 用一系列细微的凹坑压在反射表面上。要做到这一点, 首先需要用聚焦精细的高强度激光创建一个母盘, 然后再以母盘为模板制作出副本。副本有凹坑的表面被涂了一层高反射的表层, 通常是铝膜或金膜。这个闪亮的表面覆盖了一层透明丙烯酸酯以防止灰尘和划伤。最后, 在丙烯酸酯上打印上标签。

从 CD 或 CD-ROM 中检索信息是通过一个封装在光盘播放器或驱动装置中的小功率激光完成的。当电机旋转磁盘经过激光束时, 激光束射向透明的保护镀层。当激光束遇到一个凹坑时, 反射光的强度会改变。这个变化被一个光学传感器检测到, 并转换成数字信号。

回忆前面所描述的磁盘, 磁盘上的数据记录在同心的磁道上。在最简单的恒定角速度 (CAV) 系统中, 每个磁道所存储的比特位数是个常数。通过多区存储记录方式可以增大密度, 在此方式下磁盘表面被划分成很多区域, 远离磁盘中心的区域包含有比靠近中心区域更多的比特位。尽管这种技术增加了磁盘容量, 但仍然不是最佳的。

为了获得更大的磁盘容量, CD 和 CD-ROM 并没有将所保存信息组织在同心圆的轨道上。相反, 光盘上包含一个螺旋的轨道, 该轨道开始于靠近光盘中心的位置, 终止于光盘的外沿。光盘外沿的扇区与内部的扇区长度相等。这样, 信息将被均匀地封装在光盘上相同大小的段上, 而其又在光盘转速变化的情况下, 以相同的速率被扫描。因而凹坑所包含的数据被激光以恒定线速度 (CLV) 读取。光盘在访问外沿数据时的转速要慢于访问内圈数据时的转速。因而, 在靠近光盘外沿的位置, 光盘轨道的容量和旋转延迟都会增加。一张 CD-ROM 的容量大约为 680MB。

CD-ROM 适用于将大量数据分发给很多用户的情况。但考虑到在最初写数据时的开销, 其并不适用于个别应用程序。与传统的磁盘相比较, CD-ROM 有以下三个主要优势:

- 光盘的信息存储能力较大。

- 光盘及其上面的数据可以被廉价地大规模复制，而磁盘做不到这点。磁盘上的数据库只有在使用两个磁盘驱动器的情况下才能被复制到另一个磁盘。
- 光盘是可以移动的，其允许光盘本身被用来归档存储。大多数磁盘是不可移动的。在磁盘驱动器或磁盘保存新信息前，固定磁盘上存储的信息必须首先拷贝到一些其他的存储设备上。

CD-ROM 的不足如下：

- 其是只读的且不能更新。
- 访问时间远长于磁盘驱动器，最长可达半秒钟。

可记录 CD

为了满足对一个数据集只需一份或少量拷贝这种情况的需要，开发了称为可记录 CD (CD-R) 的光盘，它可以写一次，读多次。对于 CD-R 来说，光盘的制作过程如下，其中的数据可以通过一个适当强度的激光束写入。这样，通过一个比 CD-ROM 造价稍高些的光盘控制器，用户可以写一次光盘，并读取多次。

CD-R 使用的介质类似于 CD 或 CD-ROM，但并不完全相同。对 CD 和 CD-ROM 来说，信息是通过介质表面的凹坑，以改变反射率来记录的。而对 CD-R 来说，介质包括一个染色层。该染色层被用来改变折射率并且可以被高强度激光所激活。这样产生的光盘可以在 CD-R 或 CD-ROM 驱动器上读取。

CD-R 在存储文档和文件上很有吸引力。其提供了一种永久记录大量用户数据的方式。

可刻录 CD

CD-RW 光盘可以反复地写和覆盖写，如同磁盘一样。尽管尝试了很多方法，唯一被证明有吸引力的纯光学方法称为相位改变。相位改变的光盘所使用的介质，在两种不同的相位阶段会有两种显著不同的折射率。存在一个无组织的状态，在该状态下，分子展示出一种随机的方向性，反射光线能力不好；还存在一个水晶状态，在该状态下有光滑的表面，并且反射光线能力强。激光束可以将介质从一个状态转换到另一个状态。这种光盘上介质的状态改变最主要的缺陷是，介质最终会永久失去其所具有的这种特性。目前用于这种光盘的介质可以擦除 500000 到 1000000 次。

CD-RW 对于 CD-ROM 或 CD-R 来说具有明显的优势，其可以重写，因而可以用做真正的辅助存储器。正因为如此，它可以和磁盘竞争。光盘的相对于磁盘的一个主要优势在于，机械容忍度对光盘要求的严格程度远远低于对高容量磁盘的要求。光盘的一个关键优点是光盘的工程公差远不如高容量磁盘那么严格。因此，光盘展现出更高的可靠性和更长的寿命。

数字化视频光盘

由于大容量的数字化视频光盘 (DVD) 的出现，电子行业最终找到了可以替代模拟信号的家庭录像带的设备。DVD 将替代盒式录像机 (VCR) 中的录像带，甚至可以替代个人计算机和服务器中的 CD-ROM。DVD 把视频带入数字时代。它可以传送电影，保证极佳的图像质量，并且可以像音频 CD 一样自由访问，DVD 机器也能够放 CD。大量的数据被塞入 DVD 盘中，当前它的容量是 CD-ROM 的 7 倍。通过 DVD 巨大的存储能力和逼真的品质，PC 游戏将更加真实，教育软件也可以采用更多的视频。当这种材料结合到 Web 站点中，势必会对 Internet 和企业内部网的发展带来新的高峰。

DVD 具有更大的容量是由于其不同于 CD 的三个方面：

- 1) DVD 上的比特位更加紧凑。CD 上螺旋圈之间的距离为 $1.6\mu\text{m}$ ，比特位之间的距离为 $0.834\mu\text{m}$ 。DVD 使用了更短波长的激光，因此其螺旋圈之间的距离为 $0.74\mu\text{m}$ ，而比特位之间的距离为 $0.4\mu\text{m}$ 。这两方面的提高使得 DVD 的容量可以达到 4.7GB。
- 2) DVD 可以在第一层上再刻录第二层数据。一个双层 DVD 在反射层之上还有个半反射层，通过调整焦距，就可以分别读取两层数据。这种技术几乎加倍了 DVD 的容量，使其达到了 8.5GB。由于第二层较低的反射率，使其不能完全达到翻倍所应该具有的存储能力。
- 3) DVD 可以记录在光盘的两面，而不像 CD 其只能记录在光盘的一面，这样一张 DVD 最多可以容纳 17GB 的数据。

同 CD 一样，DVD 也具有只读类型和可写类型（如表 11.8 所示）。

高清晰度光盘

高清晰度光盘被设计用来存储高清晰度视频，并提供比 DVD 更大的存储容量。通过使用在蓝紫光范围内更短的激光波长，实现了更高的位密度。高清晰度光盘中构成数字 1 和 0 的数据坑与 DVD 相比更小，

这是由于其激光波长更短。

有两种相互竞争的磁盘格式和技术已经被市场所接受(见图 11.20)。高清 DVD 格式在单层单面就有 15 GB 的存储空间。对多层和双面的使用最终会导致更大的存储能力。有三个可用的类型:只读型(HD DVD-ROM)、可录制一次型(HD DVD-R)以及可重复录制型(HD DVD-RAM)。

蓝光光盘面上的数据层位置更接近激光头(显示在图 11.20 中每个图的右边)。这导致了更密集的聚焦和失真减少,从而获得了更小的数据坑和轨道。蓝光光盘的单层上可以储存 25 GB。蓝光光盘有三种类型:只读型(BD-ROM)、可录制一次型(BD-R)以及可重复录制型(BD-RE)。

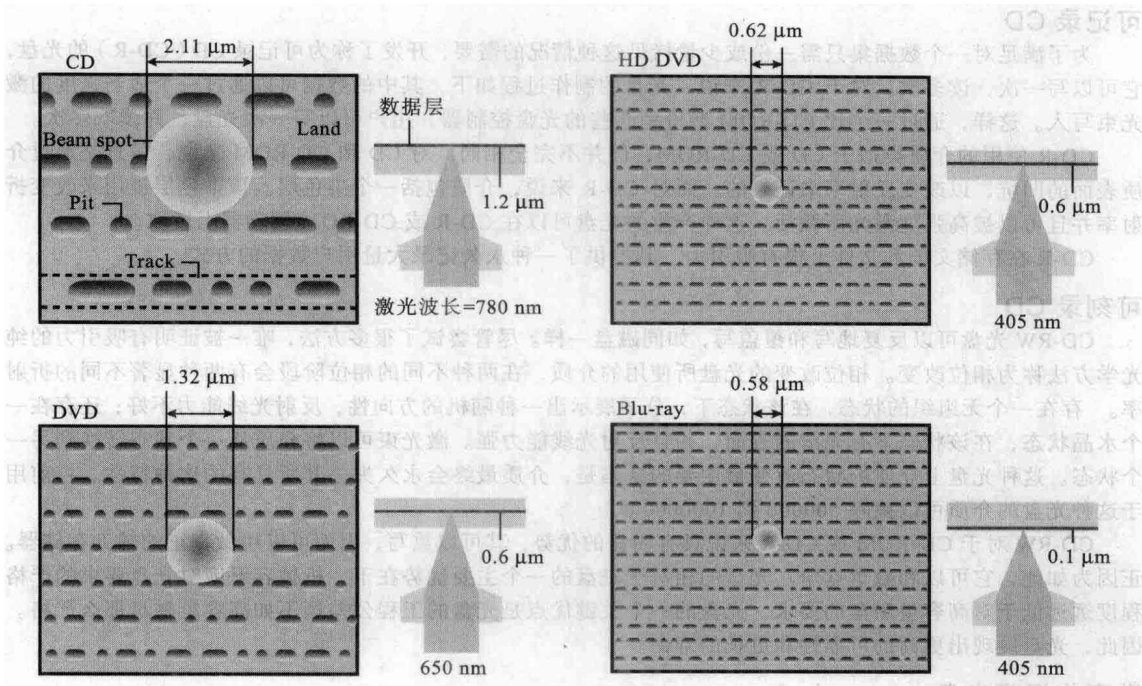


图 11.20 光存储器特性

第12章 文件管理

在大多数应用中，文件是核心部分。除了实时应用和一些特殊的应用外，应用程序的输入都是通过文件来实现的。实际上所有的应用程序的输出都保存在文件中，这便于信息的长期存储以及用户或应用程序将来访问信息。

除了把文件用作输入或输出的单个应用程序外，用户还希望可以访问文件、保存文件并保持文件内容的完整性。为实现这些目标，实际上所有的操作系统都提供了文件管理系统。在典型情况下，一个文件管理系统由系统实用程序组成，它们可以作为具有特权的应用程序来运行。一般来说，整个文件管理系统都被当做是操作系统的一部分。因此，在本书中讲述关于文件管理的一些基本原理是很有必要的。

本章首先给出一个概述，接下来介绍各种不同的文件组织方案。尽管文件的组织通常超出了操作系统的范围，但它能帮助读者理解在涉及文件管理的一些设计折衷时所做的选择。本章的其余部分讲述有关文件管理的其他主题。

12.1 概述

12.1.1 文件和文件系统

从用户的角度来看，文件系统是操作系统的一个重要部分。文件系统提供了与二级存储相关的资源的抽象。文件系统允许用户建立有所需要特性的被称为文件的数据集合，例如：

- **长期存在：**文件存储在硬盘上或其他二级存储器上。当用户注销时，文件不会丢失。
- **进程间可共享：**文件有名字，具有相关的可控制的共享访问权限。
- **结构：**通过文件系统，一个文件有对应于特定应用的内部结构。此外，文件可以被组织成层次结构或者更复杂的结构去反映文件之间的关系。

任何文件系统不但提供一个手段去存储数据（被组织为文件），而且提供一系列对文件进行操纵的功能接口。典型的操作如下：

- **创建：**定义一个新的文件，同时分配一个文件结构。
- **删除：**删除文件结构，释放相关资源。
- **打开：**一个已存在的文件由进程通过“打开”操作去打开，并允许进程对该文件进行操作。
- **关闭：**相关进程关闭该文件。这样该进程就不再能对该文件进行操作，直到进程再次打开它。
- **读：**进程读文件的所有或部分数据。
- **写：**进程更新文件，添加数据或者改变文件中已存在的数据。

一般地，文件系统为文件维护了一系列的属性。这些属性包括所有者、创建时间、最后修改时间和访问权限等。

12.1.2 文件结构

在讨论文件时通常使用域、记录、文件、数据库 4 个术语。

域（field）是基本数据单元。一个域包含一个值，如雇员的名字、日期或传感器读取的值。

域可以通过它的长度和数据类型（如 ASCII 字符串、二进制数等）来描述。域的长度可以是固定的，也可以是可变的，这取决于文件的设计。对于后一种情况，域通常包含两个或三个子域：要保存的实际值、域名，在某些情况下还包括域的长度。在其他情况下，域之间特殊的划分符号暗示了域的长度。

记录 (record) 是一组相关的域的集合，它可以看做是应用程序的一个单元。例如，一个雇员记录可能包含以下域：名字、社会保障号、工作类型、雇用日期等。同样，记录也可以是固定长度的或可变长度的，这取决于设计。如果一个记录中的某些域是可变长度的，或者记录中域的数目可变，则该记录是可变长度的。对于后一种情况，每个域通常都有一个域名。对这两种情况，整个记录通常都包括一个长度域。

文件 (file) 是一组相似记录的集合，它被用户和应用程序看做一个实体，并可以通过名字访问。文件有一个唯一的文件名，可以被创建或删除。访问控制通常在文件级实施，即在一个共享系统中，用户和程序被允许或被拒绝访问整个文件。在一些更复杂的系统中，这类控制也可以在记录级或域级实施。

有些文件系统中，文件是按照域而不是记录来组织的。在这种情况下，文件是一组域的集合。

数据库 (database) 是一组相关的数据的集合，它的本质特征是数据元素间存在着明确的关系，并且可供不同的应用程序所使用。数据库可能包含与一个组织或项目相关的所有信息，如一家商店或一项科学研究。数据库自身是由一种或多种类型的文件组成。通常数据库管理系统是独立于操作系统的。尽管它可能会使用某些文件系统管理程序。

用户和应用程序希望能够使用文件。必须支持的典型操作如下：

- **Retrieve_All**: 检索一个文件的全部记录。当应用程序在某一时刻必须处理文件中的全部信息时，需要用到这个操作。例如，产生文件的信息摘要的应用程序需要检索所有记录。由于这个操作顺序地访问所有记录，它通常等同于术语顺序处理 (sequential processing)。
- **Retrieve_One**: 仅仅要求检索一个记录。交互式的、面向事务的应用程序需要这个操作。
- **Retrieve_Next**: 要求根据最近检索过的记录，检索逻辑顺序上的下一个记录。一些交互式应用程序，如填表，可能需要这类操作，执行查找的程序也需要使用这类操作。
- **Retrieve_Previous**: 类似于 **Retrieve_Next**，但此时要求检索当前访问到的记录前面的一个记录。
- **Insert_One**: 在文件中插入一个新记录。为保持文件的顺序，新记录必须插入到文件中适当的位置。
- **Delete_One**: 删除一个已存在的记录。为保持文件的顺序，可能需要更新某些连接或别的数据结构。
- **Update_One**: 检索一个记录，更新该记录的一个或多个域，并把这个更改后的记录写回文件。同样，在这个操作中需要保持文件的顺序，如果记录的长度发生了变化，更新操作通常要更复杂一些。
- **Retrieve_Few**: 检索一部分记录。例如，应用程序或用户可能希望检索满足一些特定规则的所有记录。

这些是经常对文件执行的操作，它们会影响文件的组织方式，相关内容将在 12.2 节讲述。

需要注意的是，并不是所有的文件管理系统都呈现出在本节中讨论的这种结构。在 UNIX 或类 UNIX 系统上，文件的基本结构是字节流。例如，一个 C 程序以文件的形式存储，而没有物理域、记录等。

12.1.3 文件管理系统

文件管理系统是一组系统软件，为使用文件的用户和应用程序提供服务。在典型情况下，文件管理系统是用户或应用程序访问文件的唯一方式，它使得用户或程序员不需要为每个应用程序开发专用软件，并且给系统提供了控制最重要资源的方法。一个文件管理系统需要满足以下目标[GROS86]：

- 满足数据管理的要求和用户的需求，包括存储数据和执行上述操作的能力。
- 最大限度地保证文件中的数据有效。
- 优化性能，包括总体吞吐量（从系统的角度）和响应时间（从用户的角度）。
- 为各种类型的存储设备提供 I/O 支持。
- 减少或消除丢失或破坏数据的可能性。
- 向用户进程提供标准 I/O 接口例程集。
- 在多用户系统中为多个用户提供 I/O 支持。

关于第一点“满足用户的需求”，这些需求的范围取决于各种应用程序和计算机系统的使用环境。对于交互式的通用系统，其最小需求集合如下：

- 1) 每个用户都可以创建、删除、读取和改变文件。
- 2) 每个用户都可以受控地访问其他用户的文件。
- 3) 每个用户都可以控制允许对用户文件进行哪种类型的访问。
- 4) 每个用户都可以按照与问题相适应的形式重新构造用户文件。
- 5) 每个用户都可以在文件间移动数据。
- 6) 每个用户都可以备份用户文件，并在文件遭到破坏时进行恢复。
- 7) 每个用户都可以通过名字而不需要数字标识访问他的文件。

在关于文件系统的整个讨论中，我们都必须牢记这些目标和要求。

文件系统架构

一个了解文件管理范围的有效途径就是浏览一个典型的软件组织图，如图 12.1 所示。当然不同的系统有不同的组织方式，但这个组织具有相当的代表性。在最底层，设备驱动程序直接与外围设备（或它们的控制器或通道）通信。设备驱动程序负责启动该设备上的 I/O 操作，处理 I/O 请求的完成。对于文件操作，典型的控制设备是磁盘和磁带设备。设备驱动程序通常是操作系统的一部分。

接下来的一层称做基本文件系统或物理 I/O 层。这是与计算机系统外部环境的基本接口。该层处理在磁盘间或磁带系统间交换的数据块，因此，它关注的是这些块在二级存储和内存缓冲区中的位置，而并不关注数据的内容或所涉及的文件的结构。基本文件系统通常是操作系统的一部分。

基本 I/O 管理程序负责所有文件 I/O 的初始和终止。在这一层，需要一定的控制结构来维护设备的输入/输出、调度和文件状态。基本 I/O 管理程序根据所选择的文件来选择执行文件 I/O 的设备，为优化性能，它还参与调度对磁盘和磁带的访问。I/O 缓冲区的指定和辅存的分配也是在这一层实现的。基本 I/O 管理程序是操作系统的一部分。

逻辑 I/O 使用户和应用程序能够访问到记录。因此，基本文件系统处理的是数据块，而逻辑

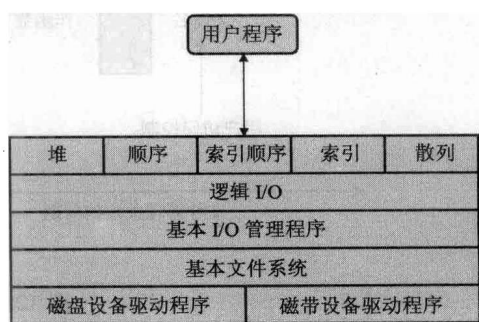


图 12.1 文件系统软件架构

I/O 模块处理的是文件记录。逻辑 I/O 提供一种通用的记录 I/O 能力，并维护关于文件的基本数据。

文件系统中与用户最近的是访问方法层，它在应用程序和文件系统以及保存数据的设备之间提供了一个标准接口。不同的访问方法反映出不同的文件结构以及访问和处理数据的不同方法。一些最常见的访问方法如图 12.1 所示，对它们的具体描述见 12.2 节。

文件管理功能

图 12.2 显示了文件系统的功能概况。我们从左到右来看这个图。用户和应用程序通过使用创建文件、删除文件以及执行文件操作的命令，与文件系统进行交互。在执行任何操作之前，文件系统必须确认和定位所选择的文件。这要求使用某种类型的目录来描述所有文件的位置以及它们的属性。此外，大多数共享系统都实行用户访问控制：只有被授权用户才允许以特定的方式访问特定的文件。用户和应用程序可以在文件上执行的基本操作是在记录级上执行的。用户和应用程序把文件看做是具有组织记录的某种结构，如顺序结构（例如，个人记录按姓氏的字母顺序存储），因此，为了把用户命令转换成特定的文件操作命令，必须采用适合于该文件结构的访问方法。

虽然用户和应用程序关注的是记录，但 I/O 是以块为基础来完成的，因此，文件中的记录必须组织成一组块的序列来输出，在输入后将各块组合起来。支持文件的块 I/O 需要许多功能。首先必须管理二级存储，包括把文件分配到二级存储中的空闲块，再者还需要管理空闲存储空间，以便知道新文件和现有文件增长时可以使用哪些块。此外，必须调度单个的块 I/O 请求，这个问题将在第 11 章讲述。磁盘调度和文件分配都影响到性能的优化，因此这些功能需要放在一起考虑。此外，优化还取决于文件结构和访问方式。因此，开发一个从性能的角度看是最优的文件管理系统是一个相当复杂的任务。

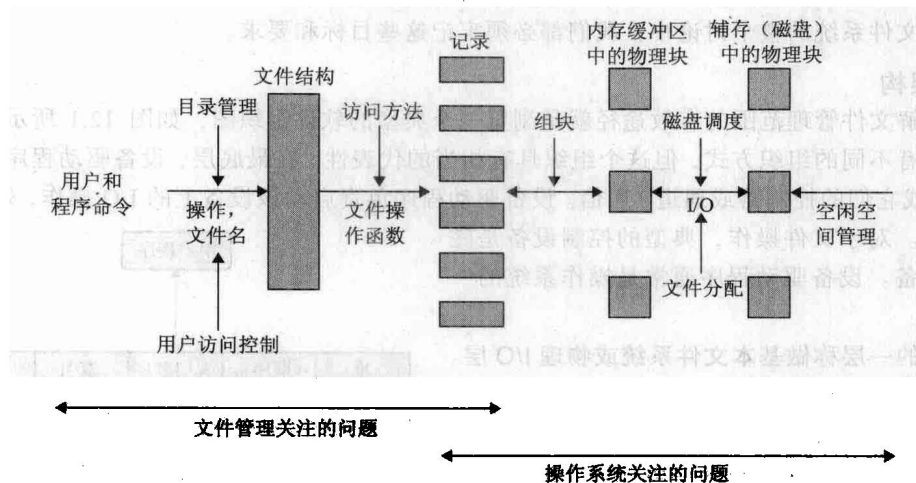


图 12.2 文件管理的要素

图 12.2 表明，文件管理系统作为一个单独的系统实用程序，和操作系统关注的是不同方面的内容，它们的交点是关于记录的处理。这个划分是任意的，不同的系统采用不同的方法。

本章的其余部分着重考虑图 12.2 中所提出的设计问题。首先讲述的是文件组织和访问方法，尽管这方面的内容已经超出了操作系统通常所考虑的范围，但是如果没有对文件组织和访问的正确评价，就不可能对与文件相关的其他设计问题进行评价；接下来阐述文件目录的概念，它们通常是由操作系统代表文件管理系统进行管理的；然后讲述文件管理的物理 I/O 特征，它们作为操

作系统设计的一个方面，其中的一个问题是逻辑记录被组织成物理块的方式；最后讲述有关二级存储中的文件分配和二级存储空间空闲空间的管理等问题。

12.2 文件组织和访问

本节使用的术语文件组织 (file organization) 指文件中记录的逻辑结构，它由用户访问记录的方式确定。文件在二级存储中的物理组织取决于分块策略和文件分配策略，这方面的问题将在本章后面的部分讲述。

在选择文件组织时，有五项重要原则：访问快速、易于修改、节约存储空间、维护简单、可靠性。

这些原则的相对优先级取决于将要使用这些文件的应用程序。例如，如果一个文件仅仅以批处理方式处理，并且每次都要访问到它的所有记录，则会很少需要关注用于检索一个记录的快速访问。存储在 CD-ROM 中的文件永远不会被修改，因此易于修改这一点根本不需要考虑。

这些原则可能是矛盾的。例如，为了节约存储空间，数据冗余应该最小；但在另一方面，冗余是提高数据访问速度的一种主要手段。这方面的一个例子是使用索引。

已经实现或刚刚提出的可选择文件组织的数目是相当多的，甚至可以编成一本专门介绍文件系统的书。在本节这个简单的概述中，主要介绍五种基本组织。实际系统中使用的大多数结构或者正好是这几类之一，或者是这些组织的组合。这五种组织为堆、顺序文件、索引顺序文件、索引文件、直接或散列文件，图 12.3 描绘了前四种。

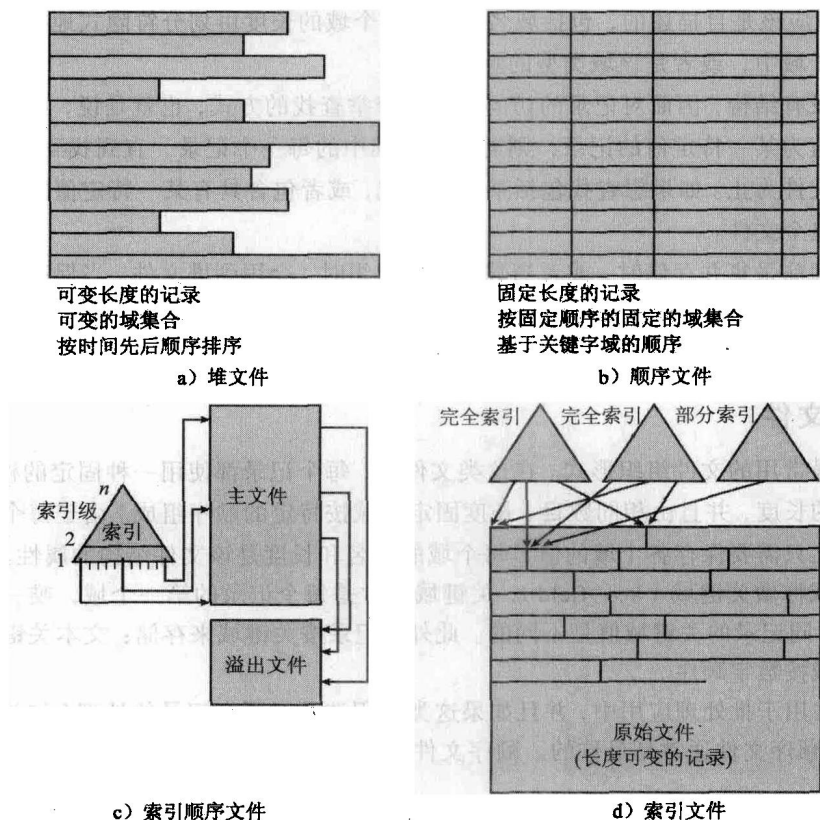


图 12.3 常用的文件组织

表 12.1 总结了这五种组织的相关性能特征[⊖]。

表 12.1 五种基本文件组织的性能等级 [WIED87]

文件方法	空 间		修 改		检 索		
	属 性		记录大小		单个记录	子集	穷举
	可变	固定	相等	大于			
堆	A	B	A	E	E	D	B
顺序	F	A	D	F	F	D	A
索引顺序	F	B	B	D	B	D	B
索引	B	C	C	C	A	B	D
散列	F	B	B	F	B	F	E

A 表示优秀，非常适合于这个目标 $\approx O(r)$
B 表示好 $\approx O(o \times r)$
C 表示足够 $\approx O(r \log n)$
D 表示需要额外的努力 $\approx O(n)$
E 表示需要特别努力才有可能 $\approx O(r \times n)$
F 表示根本不适合这个目标 $\approx O(n>1)$
其中， r 表示结果的大小， o 表示溢出的记录数， n 表示文件中的记录数。

12.2.1 堆

堆 (pile) 是最简单的文件组织形式。数据按它们到达的顺序被采集，每个记录由一串数据组成。堆的目的仅仅是积累大量的数据并保存数据。记录可以有不同的域，或者域相似但顺序不同。因此，每个域应该是自描述的，包括域名和值。每个域的长度由划分符隐式地指定，或者明确地包含在一个子域中，或者是该域类型的默认长度。

由于堆文件没有结构，因而对记录的访问是通过穷举查找的方式，也就是说，如果想找到包括某一特定域且值为某一特定值的记录，则需要检查堆中的每一个记录，直到找到想要的记录，或者查找完整个文件为止。如果想查找包括某一特定域，或者包含具有某一特定值的域的所有记录，则必须查找整个文件。

当数据在处理前采集并存储时，或者当数据难以组织时，会用到堆文件。当保存的数据大小和结构不同时，这种类型的文件空间使用情况很好，能较好地用于穷举查找，且易于修改。但是，除了这些受限制的使用，这类文件对大多数应用都是不适用的。

12.2.2 顺序文件

顺序文件是最常用的文件组织形式。在这类文件中，每个记录都使用一种固定的格式。所有记录都具有相同的长度，并且由相同数目、长度固定的域按特定的顺序组成。由于每个域的长度和位置已知，因此只需要保存各个域的值，每个域的域名和长度是该文件结构的属性。

一个特殊的域称做关键域 (key field)。关键域通常是每个记录的第一个域，唯一地标识这个记录，因此，不同记录的关键域值是不同的。此外，记录按关键域来存储：文本关键域按字母顺序，数字关键域按数字顺序。

顺序文件通常用于批处理应用中，并且如果这类应用涉及对所有记录的处理 (如关于记账或工资单的应用)，顺序文件通常是最佳的。顺序文件组织是唯一可以很容易地存储在磁盘和磁带中的文件组织。

⊖ 表中使用了大 O (大写) 表示法来描述算法的时间复杂度。附录 D 对这种表示法进行了解释。

对于查询或更新记录的交互式应用,顺序文件表现出很差的性能。在访问时,为了匹配关键域,需要顺序查找文件。如果整个文件或文件的一大部分可以一次性地取入内存,则还可能存在着更有效的查找技术。尽管如此,在访问一个大型顺序文件中的记录时,还是会遇到相当多的处理和延迟。除此之外还有一些问题。在典型情况下,顺序文件按照记录在块中的简单顺序存储,也就是说,文件在磁带或磁盘上的物理组织直接对应于文件的逻辑组织。在这种情况下,一个常用的处理过程是把新记录放在一个单独堆文件中,称做日志文件或事务文件,通过周期性地执行一个成批更新,把日志文件合并到主文件,并按正确的关键字顺序产生一个新文件。

另一种选择是把顺序文件组织成链表的形式。一个或多个记录保存在每个物理块中。磁盘中的每个块含有指向下一个块的指针。新记录的插入仅涉及指针操作,而不再要求将新记录放置在一个特定的物理块位置。因此,该方法可以带来一些方便,但它是以增加额外的处理和空间开销为代价的。

12.2.3 索引顺序文件

克服顺序文件的缺点的一种常用的方法是索引顺序文件。索引顺序文件保留了顺序文件的关键特征:记录按照关键域的顺序组织起来。但它还增加了两个特征:用于支持随机访问的文件索引和溢出文件。索引提供了快速接近目标记录的查找能力。溢出文件类似于顺序文件中使用的日志文件,但是溢出文件中的记录可以根据它前面记录的指针进行定位。

最简单的索引顺序结构只使用一级索引,这种情况下的索引是一个简单的顺序文件。索引文件中的每个记录由两个域组成:关键域和指向主文件的指针,其中关键域和主文件中的关键域相同。为查找一个特定的域,首先查找索引,查找关键域值等于目标关键域值或者位于目标关键域值之前且最大的索引,然后在该索引的指针所指的主文件中的位置处开始查找。

为说明该方法的有效性,考虑一个包含 100 万条记录的顺序文件,为查找某一特定的关键域值,平均需要访问 50 万次记录。现在假设创建一个包含了 1000 项的索引,索引中的关键域或多或少均匀分布在主文件中,为找到这个记录,平均只需要在索引文件中进行 500 次访问,接着在主文件中进行 500 次访问。查找的开销从 500 000 减少到 1000。

文件可以按以下方式处理:主文件中的每个记录包含一个附加域。附加域对应用程序是不可见的,它是指向溢出文件的一个指针。当往文件中插入一个新记录时,它被添加到溢出文件,然后修改主文件中逻辑顺序位于这个新记录之前的记录,使其包含指向溢出文件中新记录的指针。如果新记录前面的那个记录也在溢出文件中,那么修改新记录前面的那个记录的指针。和顺序文件一样,索引顺序文件有时候也会按批处理的方式合并溢出文件。

索引顺序文件极大地减少了访问单个记录的时间,同时保留了文件的顺序特性。为顺序地处理整个文件,需要按顺序处理主文件中的记录,直到遇到一个指向溢出文件的指针,然后继续访问溢出文件中的记录,直到遇到一个空指针,然后恢复在主文件中的访问。

为提供更有有效的访问,可以使用多级索引。最低一级的索引文件看做是顺序文件,然后为该文件创建高一级的索引文件。再次考虑一个包含 100 万条记录的文件,首先构造具有 10 000 项的低级索引,然后为这个低级索引构造 100 项的高级索引。查找过程从高级索引开始,找到指向低级索引的一项(平均长度=50 次访问)。接着查找这个索引,找到指向主文件的一项(平均长度=50 次访问),然后查找主文件(平均长度=50 次访问)。因此平均查找长度从 500 000 减少到 1000,最后减少到 150。

12.2.4 索引文件

索引顺序文件保留了顺序文件的一个限制:基于文件的一个域进行处理。当需要基于其他属

性而不是关键域查找一个记录时，这两种形式的顺序文件都是不能胜任的。但在某些应用中，却需要这种灵活性。

为实现这一点，需要一种采用多索引的结构，每种可能成为查找条件的域都有一个索引。索引文件一般都摒弃了顺序性和关键字的概念，只能通过索引来访问记录。其结果是对记录的放置位置不再有限制，只要至少有一个索引的指针指向这个记录即可。此外，还可以使用长度可变的记录。

可以使用两种类型的索引。完全索引中包含主文件中每条记录的索引项，为了易于查找，索引自身被组织成一个顺序文件。部分索引只包含那些有兴趣域的记录的索引项。对于长度可变的记录，某些记录并不是包含了所有的域。当往主文件中增加一条新记录时，索引文件必须全部更新。

索引文件大多用于对信息的及时性要求比较严格并且很少会对所有数据进行处理的应用程序中，例如航空公司订票系统和商品库存控制系统。

12.2.5 直接文件或散列文件

直接文件或散列文件开发直接访问磁盘中任何一个地址已知的块的能力。和顺序文件以及索引顺序文件一样，每一条记录中都需要一个关键域。但是这里没有顺序排序的概念。

直接文件使用基于关键字的散列，这项功能已在附录 8A 中描述。图 8.27b 给出了散列的组织 and 散列文件中典型使用的溢出文件的类型。

直接文件常在要求快速访问时使用，并且记录的长度是固定的，通常一次只访问一条记录，例如目录、价格表、调度和名字列表。

12.3 文件目录

12.3.1 内容

与任何文件管理系统和文件集合相关联的是文件目录，目录包含关于文件的信息，这些信息包括属性、位置和所有权。大部分这类信息，特别是与存储相关的信息，都是由操作系统管理的。目录自身是一个文件，并且可以被各种文件管理例程访问。尽管用户和应用程序也可以得到目录中的某些信息，但这通常是由系统例程间接提供的。

表 12.2 列出了目录通常为系统中每个文件保存的信息。从用户的角度看，目录在用户和应用所知道的文件名和文件自身之间提供了一种映射。因此，每个文件项都包含文件名。实际上所有系统都需要处理不同类型的文件和不同的文件组织，因此还必须提供这方面的信息。文件信息的一个重要分类涉及它的存储信息，包括它的位置和大小。在共享系统中，还必须提供用于文件的访问控制信息。典型情况下，用户是文件的所有者，可以给其他用户授予一定的访问权限。最后，还需要有使用信息，用来管理当前对文件的使用并记录文件的使用历史。

表 12.2 文件目录的信息单元

基本信息	
文件名	由创建者（用户或程序）选择的 名字，在同一个目录中必须是唯一的
文件类型	例如文本文件、二进制文件、加载模块等
文件组织	供那些支持不同组织的系统使用
地址信息	
卷	指出存储文件的设备
起始地址	文件在辅存中的起始物理地址（例如在磁盘上的柱面、磁道和块号）
使用大小	文件的当前大小，单位为字节、字或块
分配大小	文件的最大大小

(续)

访问控制信息	
所有者	被指定为控制该文件的用户。所有者可以授权或拒绝其他用户的访问，并可以改变给予他们的权限
访问信息	这个单元最简单的形式包括每个授权用户的用户名和口令
允许的行为	控制读、写、执行以及在网上传送
使用信息	
数据创建	当文件第一次放置在目录中时
创建者身份	通常是当前所有者，但并不一定必须是当前所有者
最后一次读访问的日期	最后一次读记录的日期
最后一次读的用户身份	最后一次进行读的用户
最后一次修改的日期	最后一次修改、插入或删除的日期
最后一次修改者的身份	最后一次进行修改的用户
最后一次备份的日期	最后一次把文件备份到另一个存储介质中的日期
当前使用	有关当前文件活动的信息，如打开文件的进程、是否被一个进程加锁、文件是否在内存中被修改但没有在磁盘上修改等

12.3.2 结构

不同系统对表 12.2 中的信息的保存方式也大不相同。某些信息可以保存在与文件相关联的头记录中，这可以减少目录所需要的存储量，使得可以在内存中保留所有或大部分目录，从而提高速度。当然，一些重要单元必须在目录中，在典型情况下包括名字、地址、大小和组织。

最简单的目录结构形式是一个目录项列表，每个文件一个目录项。这种结构可以用于表示最简单的顺序文件，文件名用作关键字。在一些早期的单用户系统中就已经使用了这种技术，但是当多个用户共享一个系统或者单个用户使用多个文件时，就远远不够了。

为理解一个文件结构的需求，首先考虑可能在目录上执行的操作的类型：

- **查找：**当用户或应用程序引用一个文件时，必须查找目录，以找到该文件相应的目录项。
- **创建文件：**当创建一个新文件时，必须在目录中增加一个目录项。
- **删除文件：**当删除一个文件时，必须在目录中删除相应的目录项。
- **显示目录：**可能会请求目录的全部或部分内容。通常，这个请求是由用户发出的，用于显示该用户所拥有的所有文件和每个文件的某些属性（例如类型、访问控制信息、使用信息）。
- **修改目录：**由于某些文件属性保存在目录中，因而这些属性的变化需要改变相应的目录项。

简单列表难以支持这些操作。考虑单用户的需求：用户可能有许多类型的文件，包括字处理文本文件、图形文件、电子表格等，并且用户可能希望按照项目、类型或其他某种方便的方式组织这些文件。如果目录是一个简单的顺序列表，则它对于组织文件没有任何帮助，并且强迫用户不要对两种不同类型的文件使用相同的名字。这个问题在一个共享的系统中会变得更糟。命名的唯一性成为一个严重的问题。此外，如果目录中没有内在的结构，很难对用户隐藏整个目录的某些部分。

解决这些问题的出发点是两级方案。在这种情况下，每个用户都有一个目录，还有一个主目录。主目录有每个用户目录的目录项，并提供地址和访问控制信息。每个用户目录是该用户文件的简单列表。这些方案意味着只需要在每个用户的文件集合中保证名字的唯一性，文件系统就可以很容易地在目录上实行访问限制，但是，它对于用户构造文件集合没有任何帮助。

功能更强大、更灵活的方法是层次或树状结构方法（如图 12.4 所示），这也是普遍采用的一种方法。和前面一样，有一个主目录，它的下面是许多用户目录，每个用户目录依次又有子目录

目录项和文件项，并且在任何一级都是这样。也就是说，在任何一级，一个目录都可以包括子目录的目录项和/或文件项。

目录和子目录是如何组织的将在后面阐述。当然，最简单的方法是把每个目录保存成顺序文件。当目录包含很多目录项时，这样的组织可能会导致不必要的很长的查找时间。在这种情况下，最好采用散列结构。

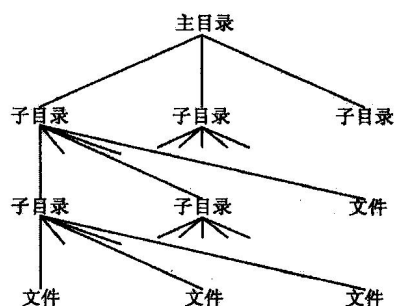


图 12.4 树状结构目录

12.3.3 命名

用户通过符号名字来引用文件。显然，为了保证文件引用无二义性，系统中的每个文件都必须具有唯一的名字。另一方面，对用户而言，要求为文件提供一个唯一的名字是一个难以接受的负担，特别是在共享系统中。

使用树状结构目录减小了提供唯一名字这方面的困难。系统中的任何文件可以按照从根目录或主目录向下到各个分支，最后直到该文件的路径来定位。这一系列目录名和最后到达的文件名组成了该文件的路径名 (pathname)。例如，图 12.5 中左下角的文件的路径名为 `/User_B/Word/Unit_A/ABC`，斜线用于划定这个序列中各个名字的界限。由于所有路径都从主目录开始，主目录名是隐含的。注意，在这种情况下，多个文件可以有相同的文件名，只要保证它们的路径名是唯一的即可。因此，系统中可以存在另外一个名为 `ABC` 的文件，但是这个文件的路径名为 `/User_B/Draw/ABC`。

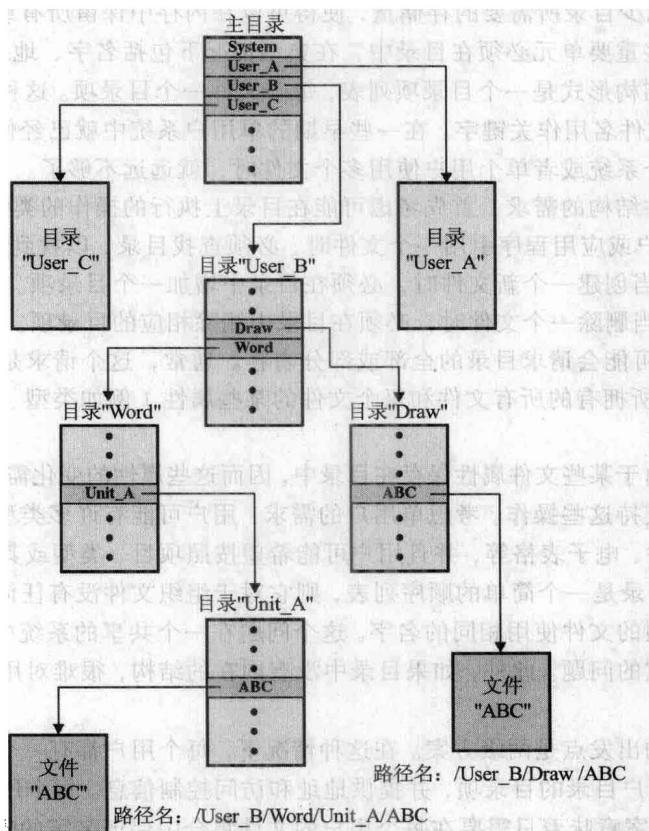


图 12.5 树状结构目录的一个例子

尽管路径名使得文件名的选择变得容易，但如果要求用户在每次访问文件时都必须拼写出完整的路径名，则还是比较难用的。典型情况下，对交互用户或进程而言，总有一个当前路径与之

相关联，通常称做工作目录（working directory）。文件通常按照相对于工作目录的方式被访问，例如，如果用户 B 的工作目录是“Word”，则路径名 `Unit_A/ABC` 足以确定图 12.5 中最左下角处的文件。当交互式用户登录进来时，或者当创建一个进程时，默认的工作目录是用户目录。在执行过程中，用户可以在树中向上或向下漫游，来定义不同的工作目录。

12.4 文件共享

在多用户系统中，几乎总是要求允许文件在多个用户间共享。这时就产生了两个问题：访问权限和对同时访问的管理。

12.4.1 访问权限

文件系统应该为允许在多个用户间广泛地共享文件提供灵活的工具。文件系统应该提供一些选项，使得访问某个特定文件的方式可以被控制。在典型情况下，用户或用户组可以被授予某些对文件的访问权限。已经使用的访问权限有很多，下面列出的是一些可以指定给某个特定用户以访问某个特定文件的具有代表性的访问权限：

- **无（none）**：用户甚至不知道文件是否存在，更不必说访问它了。为实施这种限制，不允许用户读包含该文件的用户目录。
- **知道（knowledge）**：用户可以确定文件是否存在以及其所有者。用户可以向所有者请求更多的访问权限。
- **执行（execution）**：用户可以加载并执行一个程序，但是不能复制它。私有程序通常具有这种访问限制。
- **读（reading）**：用户能够以任何目的读文件，包括复制和执行。有些系统还可以区分浏览和复制，对于前一种情况，文件的内容可以呈现给用户，但用户却没有办法进行复制。
- **追加（appending）**：用户可以给文件添加数据，通常只能在末尾追加，但不能修改或删除文件的任何内容。当在许多资源中收集数据时，这种权限非常有用。
- **更新（updating）**：用户可以修改、删除和增加文件中的数据。这通常包括最初写文件、完全重写或部分重写、移去所有或部分数据。一些系统还区分不同程度的更新。
- **改变保护（changing protection）**：用户可以改变授予其他用户的访问权限。典型地，只有文件的所有者才具备这项权力。在某些系统中，所有者可以把这项权力扩展到其他用户。为防止滥用这种机制，文件的所有者通常能够指定该项权力的持有者可以改变哪些权限。
- **删除（deletion）**：用户可以从文件系统中删除该文件。

这些权限构成了一个层次，每项权限都隐含着它前面的那些权限。因此，如果一个特定的用户被授予对某个文件的修改权限，该用户也就同时被授予以下权限：知道、执行、读和追加。

一个用户被指定成某个给定文件的所有者，通常是最初创建文件的那个用户。所有者具有前面列出的全部权限，并且可以给其他用户授予权限。访问可以提供给不同类的用户：

- **特定用户（specific user）**：由用户 ID 号指定的单个用户。
- **用户组（user group）**：不是单个定义的一组用户。系统必须可以通过某种方式了解用户组的所有成员。
- **全部（all）**：访问该系统的所有用户。这些是公共文件。

12.4.2 同时访问

如果允许多个用户追加或更新一个文件，操作系统或文件管理系统必须强加一些规范。一种蛮力的方法是当用户修改文件时，允许用户对整个文件加锁。比较好的控制粒度是在修改时对单个记录加锁。实际上，这正是第 5 章所讨论的读者-写者问题。在设计共享访问能力时必须解决互斥问题和死锁问题。

12.5 记录组块

如图 12.2 所示,记录是访问结构化文件^①的逻辑单元,而块是与二级存储进行 I/O 操作的基本单位。为执行 I/O,记录必须组织成块。

这里需要考虑以下几个问题。首先,块的长度是固定的还是可变的?在大多数系统中,块是固定长度的,这可以简化 I/O、内存中缓冲区的分配和二级存储中的块的组织。其次,与平均记录大小相比,块的相对大小是多少?一个折衷方案是,块越大,一次 I/O 操作所传送的记录就越多。如果是顺序地处理或查找文件,这显然是一个优点,因为使用大块可以减少 I/O 操作,这加速了处理。另一方面,如果是随机地访问文件,并且没有发现任何局部性,大的块会导致对并没有使用的记录的不必要的传输。但是,综合考虑顺序访问的频率和访问的局部性潜能,可以说使用大的块能减少 I/O 传送时间。需要注意的是,大块需要更大的 I/O 缓冲区,从而使缓冲区的管理更加困难。

对于给定的块大小,有三种组块方法:

- **固定组块:** 使用固定长度的记录,并且若干个完整的记录被保存在一个块中。在每个块的末尾可能会有一些未使用的空间,称做内部碎片。
- **可变长度跨越式组块:** 使用长度可变的记录,并且紧缩到块中,使得块中没有未使用空间。因此,某些记录可能会跨越两个块,通过一个指向后继块的指针连接。
- **可变长度非跨越式组块:** 使用可变长度的记录,但并不采用跨越的方式。如果下一个记录比块中剩余的未使用空间大,则无法使用这一部分,因此在大多数块中都会有未使用的空间。

图 12.6 显示了这些方法,这里假设文件保存在磁盘上的顺序块中。在图中假设文件足够大,可以跨越两个磁道。即使使用其他一些文件分配方案,其结果也不会改变(见 12.6 节)。

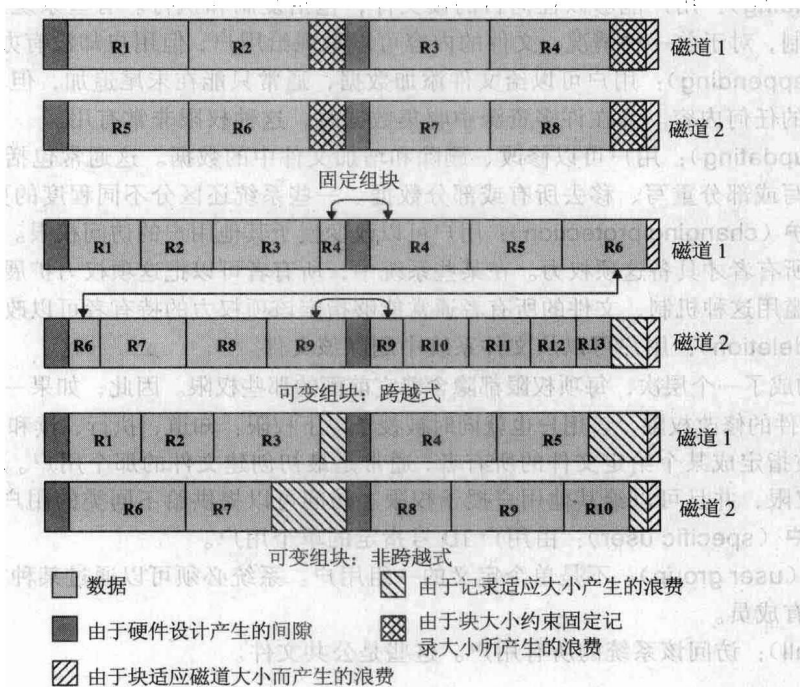


图 12.6 记录组块的方法 [WIED87]

固定组块是记录长度固定的顺序文件最常用的方式。可变长度跨越式组块的存储效率高,并

① 相对于一些文件系统中(如 UNIX 文件系统)把文件看做字节流。

且对文件大小没有限制,但是这种技术很难实现。跨越两个块的记录需要两次 I/O 操作,并且不论如何组织,文件都很难修改。可变长度非跨越式组块会导致空间的浪费,并且限制记录的大小不能超过块的大小。

如果采用记录组块技术,记录组块技术和虚存硬件会互相影响。在虚存环境中,页是传送的基本单位。页通常很小,以至对于非跨越式组块,把页当做块处理是不现实的。因此一些系统把多个页组合起来,为文件传送创建一个比较大的块。该方法已用于 IBM 主机中的 VSAM 文件。

12.6 二级存储管理

在二级存储中,一个文件是由许多块组成的。操作系统或文件管理系统负责给文件分配块。这引发了两个管理问题。首先,二级存储中的空间必须分配给文件;其次,必须知道哪些空间可以用来分配。下面将会看到,这两个问题是相关的,即文件分配采用的方法可能会影响空闲空间管理的方法。此外,文件结构和分配策略之间也是互相影响的。

本节首先讨论单个磁盘上的文件分配方法,然后讲述空闲空间的管理问题,最后讨论可靠性问题。

12.6.1 文件分配

文件分配涉及以下几个问题:

- 1) 当创建一个新文件时,是否一次性地给它分配所需要的最大空间?
- 2) 给文件分配的空间是一个或多个连续的单元,这些单元称做分区。也就是说,分区是一组连续的已经分配了的块。一个分区的大小可以从一块到整个文件。在分配文件时,分区的大小应该是多少?
- 3) 为跟踪分配给文件的分区,应该使用哪种数据结构或表?在 DOS 或其他系统中,这种表通常称做文件分配表 (File Allocation Table, FAT)。

下面依次分析这些问题。

预分配与动态分配

预分配策略要求在发出创建文件的请求时声明该文件的最大大小。在许多情况下,如程序编译、产生摘要数据文件或通过通信网络从另一个系统中传送文件时,都可以很可靠地估计这个值。但是对许多应用程序,如果不能可靠地估计文件可能的最大大小,就很难实现这种策略。在这种情况下,用户和应用程序都会多估计一些文件的大小,以避免分配的空间不够用。从二级存储分配的角度看,这显然是非常浪费的。因此,使用动态分配要好一些,动态分配只有在需要时才给文件分配空间。

分区大小

第二个问题是分配给文件的分区大小。一种极端情况是,分配一个足够大的分区,可以保存整个文件;另一种极端情况是磁盘空间一次只分配一块。因此,在选择一个分区的大小时,需要折衷考虑单个文件的效率和整个系统的效率。[WIED87] 给出了需要折衷考虑的四项内容:

- 1) 邻近空间可以提高性能,特别是对于 **Retrieve_Next** 操作,以及面向事务的操作系统中运行的事务。
- 2) 数目较多的小分区会增加用于管理分配信息的表的大小。
- 3) 使用固定大小的分区(例如块)可以简化空间的再分配。
- 4) 使用可变大小的分区或者固定大小的小分区可以减少由于超额分配而产生的未使用存储空间的浪费。

当然,这几项内容是互相影响的,必须统一考虑。其结果是可以有两种选择:

- 可变的、大规模连续分区:可以提供较好的性能。大小可变避免了浪费,并且使文件分

配表比较小,但是这又导致空间很难再次利用。

- **块:** 小的固定分区提供了更大的灵活性,但是为了分配,它们可能需要较大的表或更复杂的结构。邻近性不再是主要目标,而是根据需要来分配块。

每一种选择都适用于预分配和动态分配。对于可变的、大规模连续分区,一个文件被预分配给一组连续的块,这就消除了对文件分配表的需求,它所需要的仅仅是指向第一块的指针和分配的块的数目。一次性地分配所有分区所需要的所有块,这意味着该文件的文件分配表将保持固定大小,这是因为可以分配的块的数量是一定的。

对于可变大小的分区,我们需要考虑空闲空间的碎片问题。这个问题在第7章讨论内存的划分时已经讨论过。一些可能的选择策略如下:

- **首次适配:** 从空闲块列表中选择第一个未被使用且大小足够的连续的块组。
- **最佳适配:** 选择大小足够的未使用过的块中最小的一个。
- **最近适配:** 选择与前面分配给该文件的块组最为邻近的组,其目的是为了提局部性。

很难说哪种策略是最好的,其困难在于许多因素的相互作用,包括文件的类型、文件访问的模式、多道程序的程度、系统中的其他性能因素、磁盘缓存、磁盘调度等。

文件的分配方法

前面讨论了预分配和动态分配的比较以及分区大小等问题,现在需要考虑具体的文件分配方法。通常使用三种方法:连续、链接和索引。表12.3总结了每种方法的特点。

表 12.3 文件分配方法

	连 续	链 接	索 引	
是否预分配	需要	可能	可能	
分区大小固定还是可变	可变	固定块	固定块	可变
分区大小	大	小	小	中等
分配频率	一次	低到高	高	低
分配需要的时间	中等	长	短	中等
文件分配表的大小	一个表项	一个表项	大	中等

连续分配是指在创建文件时,给文件分配一组连续的块,如图12.7所示。因此,这是一种使用大小可变分区的预分配策略。在文件分配表中每个文件只需要一个表项,用于说明起始块和文件的长度。从单个顺序文件的角度看,连续分配是最好的。对于顺序处理,可以同时读入多个块,从而提高了I/O性能。同时,检索一个块也是非常容易的。例如,如果一个文件从块**b**开始,

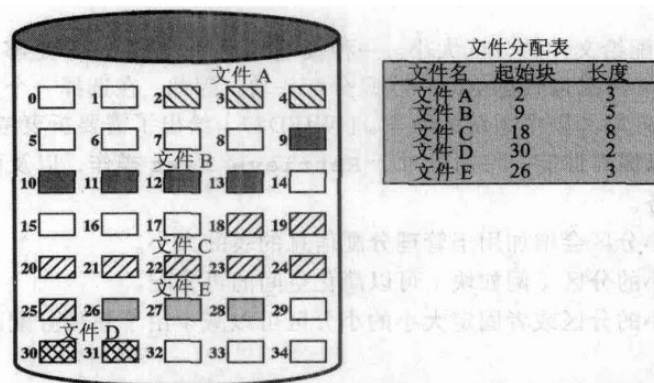


图 12.7 连续文件分配